

All Layers — Fix at the Right Layer

Your #1 mistake pattern: picking solutions at the wrong layer. This reference maps every symptom to its correct layer and correct fix.

L1 Anthropic Training · L2 System Prompt · L3 Prompt Engineering · L4 Tool Design · L5 Architecture · L6 Configuration · L7 Infrastructure

The 7 Layers — Always Fix at the Lowest Applicable Layer

The golden rule: fix at the LOWEST layer that addresses the root cause. Adding architecture complexity (L5) when a prompt change (L3) solves it = wrong layer = wrong answer.

L1 Anthropic Training (absolute limits)

Hard-coded in the model. Cannot be overridden by operators, users, or system prompts. CSAM, bioweapons, cyberweapons. Never the answer to any exam question.

FIX HERE: Nothing — these cannot be changed by anyone.

L2 Operator / System Prompt

What the operator injects at the top of every conversation. Sets persona, permissions, topic restrictions, tool availability, and default behaviours.

FIX HERE: Persona issues · wrong expertise level · topic drift · missing always-on context · behavioural framing

L3 Prompt Engineering (message content)

Instructions, examples, criteria, and structure inside each message. PRECISE framework, few-shot examples, XML tags, explicit criteria, chain-of-thought instructions.

FIX HERE: Output format inconsistency · vague criteria · missing examples → tone variance · wrong output structure · case-specific completeness gaps

L4 Tool Design (descriptions + schema)

How tools are defined — names, descriptions, input schemas, error responses. Claude uses descriptions to decide which tool to call.

FIX HERE: Wrong tool selected · tool misrouting · malformed tool calls · missing structured error metadata · overlapping descriptions

L5 Architecture (multi-agent design)

Agent structure — hub-and-spoke, coordinator/subagent roles, context passing, parallel vs sequential spawning, workflow enforcement gates, error propagation.

FIX HERE: Subagent missing info · coverage gaps · unsourced output · slow pipeline · full pipeline terminates on partial failure · loop never stops

L6 Configuration (CLAUDE.md · settings.json · MCP)

Files that define what Claude Code loads, permissions enforced, MCP servers available, and who can access shared resources.

FIX HERE: Teammate missing instructions · wrong setting wins · Bash permissions not enforced · MCP not accessible · commands not shared · context: fork issues

L7

Infrastructure (CLI flags · sessions · model)

How Claude Code is invoked — CLI flags, session naming/resuming/forking, model selection, output format, interactive vs non-interactive mode.

FIX HERE: Pipeline hangs · session lost after disconnect · context not restored · output not parseable · CI isolation broken

KEY

Always fix at the LOWEST layer that addresses the root cause. The exam rewards the minimal, precise fix — not the most sophisticated one.

Symptom → Correct Layer → Correct Fix

Symptom	Layer	Correct fix	Wrong fix (distractor)
Agentic loop never stops / stops early	L5 Arch	stop_reason == 'end_turn' ONLY	Content type check / NL parsing / iteration cap
Subagent produces wrong / incomplete output	L5 Arch	Coordinator passes complete context + structured metadata	Blame / fix subagent's system prompt
Research report misses entire topics	L5 Arch	Fix coordinator's decomposition — broader breakdown	Add more subagents (they get equally narrow briefs)
Unsourced claims in synthesis	L5 Arch	Coordinator passes metadata: URL + source + page + date	Fix synthesis subagent's prompt
Pipeline slow — sequential bottleneck	L5 Arch	Spawn independent subagents in ONE coordinator response	Faster model / reduce task scope
One failure stops entire pipeline	L5 Arch	Graceful degradation: continue others, annotate gap	Terminate all / retry indefinitely
Coordinator can't spawn subagents	L5 Arch	Add 'Agent' to coordinator's allowedTools	Fix subagent config / add subagent tools
Financial / compliance action bypasses check	L5 Arch + L4 Tool	Programmatic gate + backend validation in tool	Stronger system prompt wording
Claude calls the wrong tool	L4 Tool	Rewrite descriptions — unique, precise, non-overlapping	Add routing classifier / change tool_choice
Claude ignores MCP, uses built-in Grep	L4 Tool	Explicitly describe MCP tool's superior capabilities	Remove the built-in tool
Agent can't recover from tool error	L4 Tool	Return errorCategory + isRetryable + description	Add retry logic before fixing error structure
Downstream parser breaks on Claude JSON	L4 Tool	Tool use with forced tool_choice — guaranteed JSON	Stronger prompt: 'don't use markdown fences'
Output format / tone varies across runs	L3 Prompt	Add 2–4 few-shot examples (behavioural anchor)	More instructions / upgrade the model
Claude flags too many / too few issues	L3 Prompt	Explicit categorical criteria with concrete code examples	Adjust model sensitivity / switch models
Correct but incomplete — varies per case	L3 Prompt	Self-critique step against completeness checklist	Few-shot (fixed template can't adapt per case)
Logical errors / wrong reasoning in output	L3 Prompt	Independent review — separate Claude instance	Same-session self-critique
Claude uses wrong expertise / generic tone	L2 System	Add explicit role + persona framing in system prompt	Add to user messages (wrong layer)
Session slow / loses task after skill runs	L6 Config	Add context: fork to SKILL.md frontmatter	Split skills / summarise / faster model

Teammate doesn't get conventions / commands	L6 Config	Move from ~/.claude/ to .claude/ and commit	Tell devs to add manually
Wrong setting wins in conflict	L6 Config	Env vars > project settings.json > local > user global	Assume user-level is most authoritative
CI/CD pipeline hangs, never completes	L7 Infra	-p flag (non-interactive mode)	Plan mode / --resume / system prompt changes
Session lost after SSH disconnect	L7 Infra	--resume	New session / fork_session / invented flags
Explore 2 approaches from same baseline	L7 Infra	fork_session	--resume (same thread) / two fresh sessions
CI jobs carry over context from last run	L7 Infra	-p without --resume (fresh each time)	--resume in CI — breaks isolation

Wrong Layer Traps — Your Most Frequent Exam Mistake Pattern

These are real options that solve a real problem — just at the wrong layer. The exam builds these to catch engineers who think directionally but not precisely.

TRAP — CONTEXT POLLUTION AFTER SKILL RUNS	Wrong: Split skill into smaller pieces (reduces VOLUME)
CORRECT	Correct: context: fork in SKILL.md (changes DESTINATION)
TRAP — CLAUDE CALLS WRONG TOOL	Wrong: Add routing classifier in system prompt (L3)
CORRECT	Correct: Fix tool description (L4) — always the root
TRAP — FINANCIAL ACTION BYPASSES VERIFICATION	Wrong: Stronger system prompt warning (L3 — probabilistic)
CORRECT	Correct: Programmatic gate + backend validation (deterministic)
TRAP — OUTPUT VARIES IN TONE	Wrong: More explicit instructions (wrong L3 sub-technique)
CORRECT	Correct: Few-shot examples (correct L3 sub-technique — behavioural anchor)
TRAP — CORRECT BUT INCOMPLETE PER CASE	Wrong: Few-shot for known case types (fixed template)
CORRECT	Correct: Self-critique step (adapts to each specific response)

TRAP — LOGICAL ERRORS IN OUTPUT	Wrong: Self-critique same session (same reasoning approves errors)
CORRECT	Correct: Separate Claude instance with fresh context
TRAP — TEAMMATE MISSING CONVENTIONS	Wrong: 'Tell them to add to local config'
CORRECT	Correct: Move to project-level, commit to git
TRAP — CI PIPELINE HANGS	Wrong: Add plan mode (requires interactive approval — hangs)
CORRECT	Correct: -p flag (non-interactive)
TRAP — COVERAGE GAP IN RESEARCH	Wrong: Add more subagents (get equally narrow briefs)
CORRECT	Correct: Fix coordinator's decomposition — broader breakdown
TRAP — CUSTOMER UPSET — AGENT STRUGGLING	Wrong: Escalate (wrong trigger — sentiment ≠ policy gap)
CORRECT	Correct: Solve the problem — frustration is not an escalation trigger
KEY	The 5-second test: does the fix operate at the SAME layer as the symptom? If not — it's the wrong answer, regardless of how reasonable it sounds.

Layer 3 Deep Dive — Which Prompt Technique Fixes Which Problem

Technique	Fixes	Does NOT fix	Exam signal words
Few-shot examples (2–4)	Format/tone/length inconsistency. Wrong approach/pattern chosen.	Adaptive completeness gaps. Logical errors.	'varies in format', 'inconsistent structure', 'wrong tone'
Explicit criteria	Vague instructions → inconsistent quality. False positives from 'be conservative'.	Format variance (need examples). Missing content.	'sometimes flags', 'inconsistent quality', 'be thorough isn't working'
Self-critique step	Case-specific completeness gaps — correct but missing policy/timeline/next steps.	Logical errors (same reasoning approves them). Format variance.	'sometimes omits', 'inconsistently explains', 'varies by case'
XML tags	Claude confusing content with instructions. Complex multi-input prompts.	Quality. Tool misrouting. Session management.	'injection risk', 'content mistaken for instructions'

Chain of thought	Wrong answers on complex reasoning tasks. Claude jumps to conclusions.	Format issues. Tool selection. Session problems.	'reasoning errors', 'wrong conclusions on complex tasks'
Role / persona framing	Wrong expertise level. Generic responses lacking domain depth.	Tool misrouting. Config problems. Session issues.	'generic tone', 'wrong expertise', 'not domain-appropriate'

Layer 7 Quick Reference — Real CLI Flags vs Invented Distractors

■ Invented flags — eliminate instantly

- --recover last ← Q mistake
- --restore
- --reconnect
- --continue
- --reload
- --checkpoint
- --replay
- --sync
- --persist
- .claude_history file

■ Real flags — memorise these

- --name start named persistent session
- --resume rehydrate persisted session after disconnect
- -p / --print non-interactive CI/CD mode
- --output-format json machine-parseable output
- fork_session branch from checkpoint, diverge
- /memory debug what files are loaded
- /compact compress conversation history
- /clear reset context entirely

Scenario	Correct flag	Wrong options to eliminate
SSH drops — restore 20 turns of context	--resume	New session / fork_session / any invented flag
Start a session you plan to resume later	--name	Any flag that implies resuming — this is the START
Explore 2 approaches from same baseline	fork_session	--resume (same thread) / two fresh sessions
CI/CD automated pipeline	-p --output-format json	--resume (breaks isolation) / plan mode (hangs)
Context window filling up	/compact (in-session)	New session (loses context) / --resume
Claude ignoring CLAUDE.md	/memory (in-session)	Rewriting the file without checking if it loads

All Layers reference complete. For enforcement guidance, see CCA-F_Anthropic_Guidance_Principles.pdf